This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Decentralized Digital Identity Management for Large Language Model Agents

Abdullah Aydeger, Engin Zeydan, Josep Mangues-Bafalluy, Yekta Turk, Sunder Ali Khowaja, and Kapal Dev

Abstract

A Large Language Model (LLM) agent is an AI system that uses an LLM as its core to interact with its environment and perform tasks that go beyond text generation by integrating reasoning, planning, and external tools. While these agents automate complex language-based tasks, they face security risks such as prompt injection attacks, unauthorized access, and exploitation by external Application Programming Interfaces (APIs). This paper explores integrating LLMs with blockchain-based Self-Sovereign Identity (SSI) systems to enhance digital governance in AI applications. Experimental results show that integration can improve security, achieving up to 100% validation success at low load and 93% at peak load surpassing traditional authentication systems. However, improved security comes with an increased cost in terms of latency and throughput, which requires further investigation into optimizations such as Layer 2 solutions for scalability.

Introduction

The rapid development of Artificial Intelligence (AI) and blockchain technology has opened up new avenues for the development of secure, efficient and intelligent systems. Among these advances, foundation models such as Large Language Models (LLMs), e.g., OpenAI's GPT-4, have shown exceptional capabilities in understanding and generating human-like content [1]. LLM agents that utilize advanced natural language processing can automate a wide range of tasks, from simple data retrieval to sophisticated decision-making, and these agents go beyond simple automation. They integrate advanced processing, dynamic scheduling, and seamless interaction with various digital tools, making them indispensable for solving complex, multi-layered problems.

Emerging platforms emphasize the growing role of LLM agents in AI-driven solutions by improving efficiency, adaptability and robustness. Despite their advantages, the increasing use of LLM agents raises significant security concerns, particularly with regard to the interactions between these agents. One of the major problems is that malicious organizations can exploit vulnerabilities in communications or manipulate the behavior of agents for adversarial purposes. One example is a prompt injection attack, where agents are deceived into disclosing confidential information or performing unintended actions through manipulated inputs. In addition, external dependencies on APIs and third-party tools increase vulnerability to unauthorized access and potential command execution threats. Furthermore, inherent weaknesses of LLMs such as hallucination (i.e., misrepresentation or fabricated information) and bias can lead to unpredictable and even harmful consequences in agent interactions. The lack of standardized security frameworks and assessment methods exacerbates these risks and makes LLM agents vulnerable to exploitation.

To mitigate these risks, this paper proposes to integrate LLM agents into blockchain-based Self-Sovereign Identity (SSI) systems to strengthen security and improve AI-driven application management. The major contributions of this work are as follows:

- A secure and layered architectural framework enabling decentralized authentication, privacy-preserving interactions, and verifiable agent collaboration without relying on centralized trust anchors.
- A functional taxonomy of LLM-agent interfaces, including Interaction, Data Management, Security, and Integration layers, which serve as design blueprints for real-world implementations of agentic AI systems.
- An experimental evaluation using Hyperledger Indy and simulated agent workflows, which demonstrates strong security improvements (up to 100% validation success at low load) and quantifies performance trade-offs (e.g., latency and throughput under increasing agent load).

LLMs, AI Agents, and Blockchain SSI

LLMs, such as OpenAI's GPT series, have made significant advances in natural language understanding and generation. These models have been

Abdullah Aydeger (corresponding author) is with Florida Institute of Technology, Melbourne, FL 32901 USA; Engin Zeydan and Josep Mangues-Bafalluy are with the Centre Tecnològic de Telecomunicacions de Catalunya (CTTC), 08860 Barcelona, Spain; Yekta Turk is with Aselsan Corporation, 34906 Istanbul, Türkiye; Sunder Ali Khowaja is with the CONNECT Centre and the School of Computing, Dublin City University, Dublin, D09 V209 Ireland; Kapal Dev is with the ADAPT Centre, Department of Computer Science, Munster Technological University, Bishopstown, Cork, T12 P928 Ireland, also with the Institute of Intelligent Systems, University of Johannesburg, Auckland Park, Johannesburg 2006, South Africa, and also with the Centre for Research Impact and Outcome, Chitkara University Institute of Engineering and Technology, Chitkara University, Rajpura, Punjab 140401, India.

Digital Object Identifier: 10.1109/MCOMSTD.2025.3648666

2471-2825 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Aspect	Traditional Centralized	Federated Identity Systems	Blockchain-Based SSI
Trust Model	Central server	Identity Providers (IdPs)	Peer-to-peer, decentralized
Identity Verification	Server-controlled credentials	Token-based authentication	Verifiable credentials (VCs) signed on blockchain
Resilience to Attacks	Vulnerable to breaches, phishing	Improved but dependent on IdPs	Highly resilient; cryptographic proofs
Auditability	Low; logs controlled by server	Moderate; limited transparency	High; tamper-proof blockchain records
Interoperability	Platform-specific	Cross-platform with IdP integration	Global interoperability with DIDs/VCs
Scalability	High, but bottlenecks exist	Depends on IdP capacity	Requires optimization (e.g., Layer-2)

TABLE 1. Comparison of Authentication Methods for LLM Agents.

extensively studied for their capabilities in various domains, such as text summarization, question answering, and conversational AI [2]. The use of LLMs in autonomous agents has been explored for tasks such as planning, problem-solving, and tool usage, highlighting their potential to automate complex workflows [3], [4]. The architecture of AI agents using LLMs often includes components such as memory management, task decomposition, and dynamic scheduling. Internet of Agents (IoA) framework has been proposed in [5], which introduces an agent integration protocol, an instant messaging-like architectural design, and dynamic mechanisms for agent teaming and conversation flow control. However, existing IoA frameworks and LLM-based agent architectures typically overlook secure and decentralized authentication mechanisms, especially those based on blockchain technologies.

Securing LLM agents presents unique challenges. Traditional security models rely heavily on centralized identity providers and access control mechanisms, which introduce risks such as single points of failure, identity spoofing, and server-side breaches. Moreover, LLM agents are vulnerable to novel attack vectors, including prompt injection attacks, adversarial prompting, model bias exploitation, and unauthorized API access. Studies have shown that LLMs can be manipulated into leaking sensitive information or performing unintended actions when presented with carefully crafted inputs [6]. These vulnerabilities are exacerbated in multi-agent systems, where agents dynamically interact without robust identity verification frameworks. While efforts have been made to strengthen the security of LLM agents through federated authentication and API-based access controls [7], these methods still suffer from inherent limitations such as (i) Reliance on centralized servers creates bottlenecks and failure points, (ii) Current methods often lack cryptographically verifiable identity proofs, (iii) Traditional systems do not provide immutable records of authentication events, (iv) Identity credentials are siloed and platform-dependent.

Blockchain-based SSI offers a promising alternative by enabling decentralized, user-controlled identity management. In SSI systems, users own and manage their credentials, which are stored off-chain and cryptographically verifiable using blockchain-based decentralized identifiers (DIDs) and verifiable credentials (VCs) [8]. This architecture provides strong guarantees of authenticity, non-repudiation, and privacy-preserving selective disclosure. Blockchain technology further enhances security through tamper-proof, transparent records of credential issuance, updates, and revocation [9]. Recent studies have explored the potential of combining blockchain and AI to secure data access and manage trust among autonomous agents [10], [11]. However, there remains a significant gap in the literature regarding (i) How LLM agents can dynamically authenticate user and agent identities using decentralized verification mechanisms? (ii) How blockchain-based SSI can mitigate security vulnerabilities unique to multi-agent LLM systems?, (iii) How to systematically design agent workflows that integrate decentralized authentication, authorization, and auditing?

In this work, we address these gaps by proposing a framework in which LLM agents leverage blockchain-based SSI for secure identity management, decentralized trust establishment, and privacy-preserving interactions. The integration enables agents to dynamically verify each other without relying on centralized intermediaries, improving the resilience, auditability, and scalability of AI-driven applications. Table 1 provides a comparison of traditional, federated, and blockchain-based SSI authentication methods for LLM agents.

LLM Agents

LLM agents are specialized AI systems that combine planning, memory, and tools to tackle complex problems. These agents are capable of breaking down tasks into manageable sub-tasks, using both short-term and long-term memory to improve their performance over time. LLM agents use short-term memory to manage immediate tasks and long-term memory to store information from past interactions so that they can continuously learn and adapt. They can be used to formulate and reflect on plans and adjust their strategies based on real-time feedback and results. Finally, LLM agents can be integrated with various tools and APIs, enhancing their ability to perform specialized tasks such as data retrieval, coding, and real-time analysis.

LLM Categorization

Figure 1 shows six transformative AI agents recently introduced by Google. *Customer agents*

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

engage in human-like conversations, understand needs, and give tailored recommendations via chatbots, virtual assistants, or in-car systems (e.g., Mercedes-Benz, Samsung). *Employee agents* automate workflows, analyze documents, and boost productivity, as used by Uber for summarizing interactions. *Creative agents* produce and edit images, videos, audio, and text to brand guidelines, including AI-powered workspace apps with auto-generated videos. *Data agents* analyze large datasets, build forecasts, detect anomalies, and offer recommendations using tools like BigQuery and Vertex AI. *Code agents* assist developers with writing, debugging, and optimizing code. *Security agents* monitor systems, detect threats, and protect sensitive data in real-time.

Agentic AI

Agentic AI refers to artificial intelligence systems that utilize LLM agents as a core component to exhibit agency, i.e. autonomy, proactive goal pursuit and adaptive decision-making. In contrast to passive tools using AI, Agentic AI systems dynamically decompose tasks, self-orchestrate workflows and interact with tools or external environments without continuous human intervention. In the proposed work, Agentic AI integrates LLM agents with planning, memory and tool utilisation capabilities to solve complex problems such as real-time threat response. The basis of Agentic AI setup in our experiments corresponds to a system in which LLM agents initiate actions, evaluate outcomes, and adapt strategies based on environmental feedback.

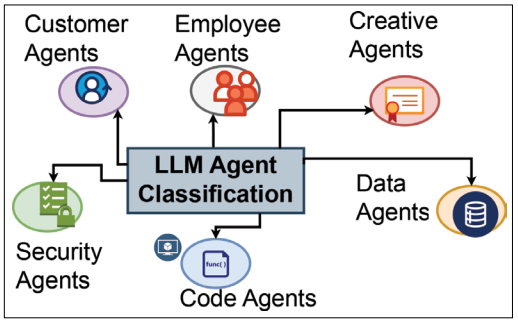


FIGURE 1. LLM Agent Classification.

Threat Model and Adversarial Scenarios

LLM agents operating in decentralised environments face a wider range of threats that go beyond the traditional vulnerabilities of centralised systems. Agent-specific threats include impersonation attacks, where malicious entities create fake agent identities to infiltrate multi-agent collaboration, and cross-agent manipulation where compromised agents attempt to influence legitimate agents through manipulated interactions or data poisoning. In cooperative multi-agent scenarios, attackers can deploy malicious agents that appear to make a positive contribution but gradually compromise the group's decision-making processes.

Blockchain-specific threats to identity systems include Sybil attacks, where adversaries create multiple fake identities to overwhelm verification systems, and potential 51% attacks on smaller permissioned networks that manage agent credentials [12]. Smart contract vulnerabilities in credential management could allow unauthorised modification or revocation of credentials. In addition, privacy threats arise from blockchain

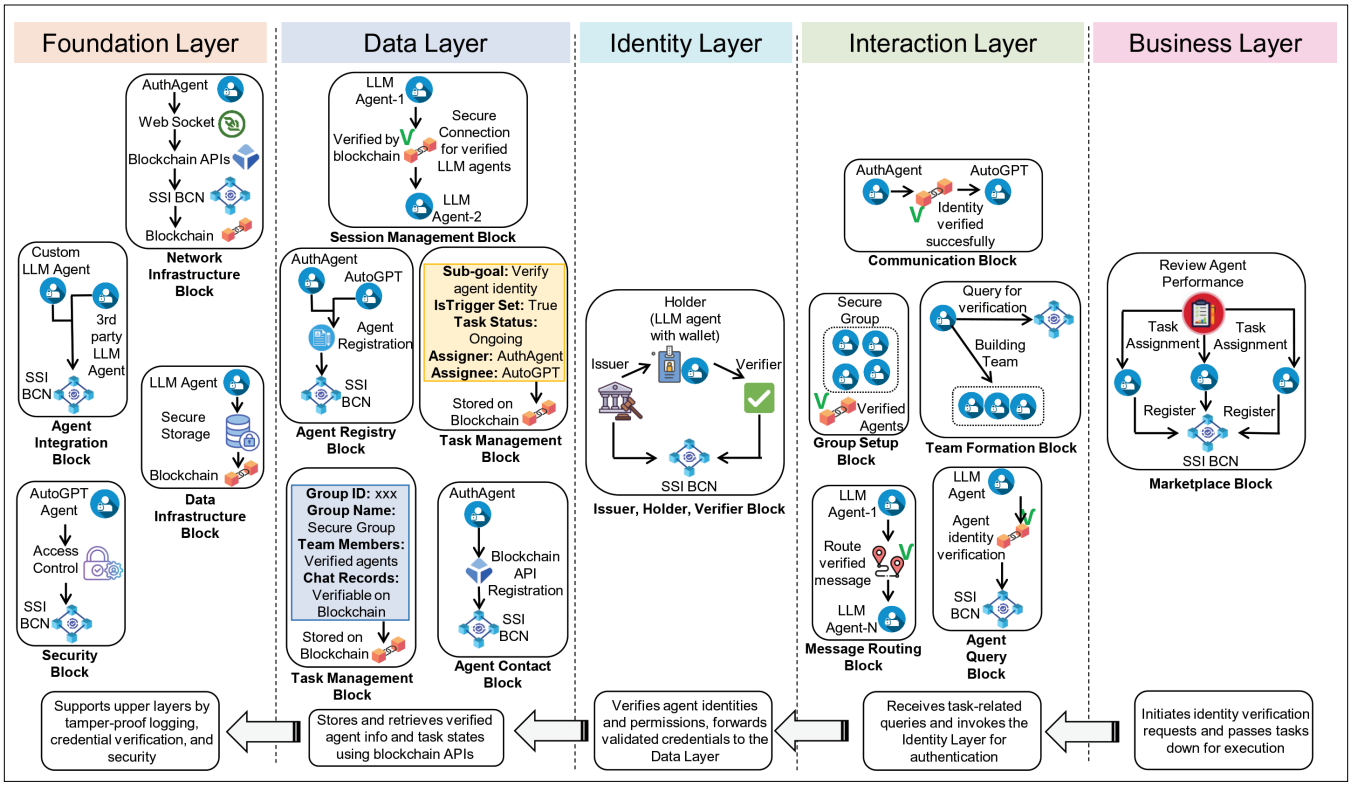


FIGURE 2. The integration of LLM agents with blockchain-based SSI in multiple layers.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

transparency, as correlation attacks can link agent interactions across transactions to de-anonymise agents, and analysis of communication patterns could reveal sensitive operational information. Sophisticated hybrid attacks can combine AI and blockchain vulnerabilities, e.g. using prompt injection to trick agents into accepting fraudulent credentials or performing unauthorised blockchain transactions. Our blockchain-based SSI approach addresses these threats through cryptographically verifiable credentials, immutable audit trails that detect tampering attempts, and decentralised verification that eliminates single points of failure present in centralised identity systems.

System Architecture

Figure 2 illustrates the integration of LLM agents with blockchain-based SSI systems, which is an extended version of [5], and is divided into five main layers: Business Layer, Interaction Layer, Identity Layer, Data Layer, and Foundation Layer.

In the business layer, the LLM agent marketplace uses blockchain-based SSI for discovery and matchmaking based on verified skills. Agents register immutable credentials and capabilities on the blockchain, enabling precise task assignment. Ratings and performance records stored on-chain ensure transparent selection, e.g., in e-commerce, agents for payments or order management can be chosen based on verified histories.

In the interaction layer, the team formation block is responsible for verifying agent identities by querying credentials and interacting with the blockchain. This involves agents such as AutoGPT, which use the blockchain node to securely authenticate team members. In the communication block, agents such as AuthAgent and AutoGPT carry out secure communication, whereby messages confirming successful identity verifications are logged in accordance with the protocol. The agent query block processes identity verification queries and accesses the blockchain to authenticate agent credentials. The group setup component sets up groups of verified agents and ensures that only authenticated members are included. Message routing manages the routing of verified messages between team members and ensures secure communication channels.

In the identity layer, blockchain-based SSI can facilitate secure and efficient interactions between agents and between agents and humans by providing a robust authentication and authorization framework. When agents need to collaborate, they can authenticate each other using blockchain-based SSI, ensuring secure and trusted communications. When an LLM agent receives a complex task, the identity layer can use blockchain-based SSI to dynamically verify the identities and authorizations of all agents involved. This prevents unauthorized agents from accessing sensitive data or performing critical tasks.

In the data layer, within the agent contact block, agents use tools such as the blockchain API to verify credentials and establish secure connections. Group info block manages records of group information, including verified members and chat records, all of which are verifiable on the blockchain. The task management block manages tasks such as verifying agent identities, tracking status, and assigning these tasks. Agent registry block lists agents together with their tools and capabilities, verified by the blockchain. Session management block manages secure WebSocket connections for verified agents, ensuring continuous secure sessions.

In the foundation layer, the agent integration block connects custom and third-party agents to blockchain-based authentication. The data infrastructure block securely stores and exchanges interaction data, with blockchain-based SSI ensuring tamper-proof logs for accountability and auditability. Verified user preferences and histories can support personalized services without compromising privacy. The network infrastructure block enables secure real-time communication, while the security block enforces access control and data integrity via blockchain verification.

LLM Agent and Blockchain-Based SSI Interfaces

LLMs are poised to play a major role across domains and may surpass human capabilities, yet they still face challenges in tasks involving collaboration, workflow integration, and tool usage. Well-designed interfaces can bridge this gap and enhance multi-agent success. We propose four categories of such interfaces: Interaction, Data Management, Security, and Integration.

1. Interaction Interfaces: These interfaces facilitate communication and collaboration between LLM agents and other entities, including humans and other agents. For example, an LLM agent integrated into a project management tool can automatically update the status of tasks, assign new tasks based on project progress and notify team members of deadlines and changes. Their components are: (i) Natural Language Processing (NLP) Application Programming Interfaces (APIs) enable LLM agents to understand and generate human language to enable seamless communication. (ii) Collaboration tools interfaces that integrate with collaboration platforms such as Slack, Microsoft Teams or GitHub to enable LLM agents to participate in team discussions and project management. (iii) Task management systems interfaces that connect LLM agents with task management tools (e.g. JIRA, Trello) to dynamically assign, track and update tasks. An *Example Interaction Flow:* LLM agent analyzes project progress → Generates task update → Calls Jira REST API endpoint to create/update a ticket.

2. Data Management Interfaces: These interfaces ensure the secure and efficient management of data that LLM agents need to perform their tasks. For example, an LLM agent in an online education platform can access a student's learning history and preferences stored on a blockchain, ensuring data privacy and personalized learning experiences. Their components are: (i) Blockchain data interfaces provide access to immutable and verifiable data stored in blockchain ledgers, ensuring data integrity and transparency. Allow agents to fetch/verifiy credential data from SSI registries (e.g., Hyperledger Indy Pool). (ii) Database APIs enable LLM agents to query and manipulate data stored in SQL/NoSQL databases. (iii) Data privacy tools interfaces manage access control and data privacy and ensure that LLM agents only access relevant and authorized data. Incorporate

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Attribute-Based Access Control (ABAC) and decentralized identity-based policies to ensure data minimization principles. *Blockchain Query for Credentials Example:* Below snippet demonstrates an LLM agent querying the Indy ledger to fetch a credential definition needed for secure interaction.

```
from indy import ledger, did
# Retrieve Verifiable Credential
get_cred_def_request = await led-
ger.build_get_
cred_def_request(
     did_user, cred_def_id
)
cred_def_response = await ledger.
submit_request(
   pool_handle, get_cred_def_request
)
```

3. Security Interfaces: These interfaces ensure the security and integrity of interactions and data handled by LLM agents. For example, in a financial services application, an LLM agent providing financial advice can verify the credentials of a payment processing agent using blockchain-based SSIs, ensuring secure and trustworthy collaboration. Their components are: (i) Authentication and authorization APIs use blockchain-based SSI to verify the identity and permissions of users and agents before granting access to resources. (ii) Encryption tools ensure that all communication and data storage is encrypted to protect against unauthorized access. For privacy-preserving, Zero-Knowledge Proofs (ZKP) (e.g., zk-SNARKs, zk-STARKs) are employed to verify credentials without revealing underlying confidential information. (iii) Audit logs maintain an immutable record of all interactions and transactions conducted by LLM agents to ensure accountability and compliance. Agents write tamper-evident logs to an append-only ledger or Interplanetary File System (IPFS), capturing events such as authentication, task delegation, and message routing. *Example Zero-Knowledge Proof Authentication Flow:* (i) LLM agent receives a proof request from another agent (e.g., financial service agent). (ii) Uses Indy SDK's `prover_create_proof ()` function to generate a zk-proof. (iii) Sends proof without revealing private credential attributes.

4. Integration Interfaces: These interfaces allow LLM agents to connect to various tools and systems required to perform tasks. For example, an LLM agent involved in software development can use tool adapters to integrate with IDEs, version control systems and CI/CD pipelines to automate code generation, testing and deployment. Their components are: (i) API gateways facilitate integration with external APIs and enable LLM agents to access a wide range of services and data allowing agents to access external APIs, managing authorization tokens, rate limits, and error handling. (ii) Tool adapters offer specific interfaces for integration with development tools, analysis platforms and other specialized software. They enable connections to developer environments (e.g., VS Code plugin adapters, GitHub Actions API) or CI/CD pipelines. (iii) Workflow orchestration engines manage complex workflows by coordinating the actions of multiple LLM agents and tools. Engines such as Apache Airflow, Prefect, or custom FSMs manage multi-step workflows involving multiple LLM agents. *Example Agentic Orchestration:* A Workflow Manager LLM agent can (i) Fetch a task list from Jira. (ii) Invoke an AuthAgent to validate the identity of a contributing LLM agent. (iii) Trigger a Deployment Agent to execute builds and test runs through a secured GitHub Actions API.

Interface Type	Core Technologies
Interaction Interfaces	NLP APIs (OpenAI, HuggingFace), Webhooks (Slack, Jira), REST APIs
Data Management Interfaces	Indy SDK, SQL/NoSQL databases, ABAC policies
Security Interfaces	AES-256 encryption, DIDComm 2.0, zk-SNARKs proofs, audit logs on IPFS
Integration Interfaces	API Gateway, Docker API, GitHub API, Apache Airflow

TABLE 2. LLM Agent Interface Types and Core Technologies.

A summary of the interface types and core technologies of LLM agents is listed in Table 2. Note that the interface descriptions contained in this section are for illustrative purposes only and serve as a design template. Quantitative performance analyses of these interface modules, such as throughput, latency or protocol efficiency, are reserved for future empirical studies once full implementations.

Experimental Analysis

In this section, we present the experimental analysis of the proposed work by integrating agents based on LLMs into a blockchain-based SSI system. The experimental setup was developed using Hyperledger Indy and the comparison is made with a traditional non-AI agent based centralized authentication system. The evaluation compared an Agentic AI system, leveraging blockchain-based SSI and LLM agents, against a centralized non-Agentic AI system based on traditional password-based authentication with multi-factor authentication (MFA). The choice of a password/MFA baseline was intentional for two reasons. First, it provides a simple, widely deployed security standard suitable for controlled experimental evaluation. Second, it offers a clear view of the security improvements achievable through blockchain-based decentralized authentication.

Experimental Setup

We utilized two different configurations for the deployment used for experiments. The first configuration corresponds to an Agentic AI system that uses two virtual machines. The first virtual machine was hosted on a REST server implemented with FastAPI/Flask in Python. The server was used to manage the interactions of the LLM agent within the Docker containers. The second virtual machine was operated via a Hyperledger Indy node for decentralized identity management. A local pool ledger node connection was initialized with a genesis transaction file using IndySDK for verification and DID creation. The software stack for the experiments includes Docker, Python, Hyperledger Indy and OpenStack (Yoga Release). The non-agent AI system, on the other

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

hand, was deployed on a single virtual machine with Apache web server and MySQL for credential storage. Python was used in this setup for processing requests and simulating a centralized password-based authentication system with MFA.

The workflow has been divided into four phases, i.e., Identity Creation and Management (ICM), Secure Interaction (SI), Transaction Authorization (TA) and Performance Evaluation (PE). The ICM registers credentials in MySQL for both DIDs generated by the AI system via Indy and those generated by the non-AI system. The SI tests interactions with AI agents in cooperation mode and compares them with a non-Agentic AI system using user-to-server authentication. The TA authorizes data access and financial transfers for non-Agentic AI systems with password/MFA and Indy's Verifiable Credentials (VCs) for Agentic AI systems. Finally, the PE evaluates uptime, resource utilization, error rates, security validation, throughput and latency for loads of 10, 50, 100, 250 and 500 agents/users. ThreadPoolExecutor from Python was used to perform the stress test for parallel operations. The asyncio was used to manage the interactions of Indy in an asynchronous way. We extend the baseline analysis with a federated identity system (OAuth 2.0), deployed via Keycloak. This setup uses Python's oauthlib for token-based authentication and thus simulates enterprise-grade identity providers (IdPs). Agent interactions require OAuth access tokens, with authorisation policies mirroring the SSI/non-Agentic AI workflows.

Experimental Results

The results are shown in Table 3 using varying, increasing loads with both Agentic AI and non-Agentic AI systems. From the experimental results, it can be deduced that the latency in identity creation with Agentic AI increased from 150 ms to 600 ms when the load was varied from 10 agents to 500 agents, while the verification latency also increased from 100 ms to 480 ms at the same load. The latencies indicate a significant blockchain overhead when derived from real Indy ledger operations. However, security improves when using Agentic AI, as shown by the security validation success rate. The success rate remained high, i.e. 100% at low load, and only decreased by 7% when 500 agents were deployed. The results indicate that the Agentic AI setup blocked 465 out of 500 unauthorized access attempts at peak load. Indy's processing limitations were evident in the transaction throughput for data access and financial transfers, which dropped from 24 transmissions/second to 5 transmissions/second and from 19 transmissions/second to 3 transmissions/second at full utilization. Error rates also increased from 0 to 6% and resource utilization increased to 95% CPU at full load. Similarly, when deploying 500 agents, memory usage increased to 98%, dropping uptime to 95%.

In contrast, the non-Agentic AI system exhibited lower latency, i.e., 30 – 50 ms for verification and 50 – 80 ms for creation, and higher throughput, i.e., 35 – 20 transmissions/second for financial transfers and 40 – 25 transmissions/second for data access, which is an implicit advantage for using a centralized system. However, the security validation success dropped from 98% to 80% when the load was varied, indicating that only 400 out of 500 unauthorized attempts were blocked. The security validation success rate highlights the vulnerability of the non-Agentic AI system. Subsequently, the error rates increased from 1% to 7% while the load was varied. Resource utilization remained at 30% for memory and 25% for CPU. The uptime dropped to 95% under stress. The higher security level is achieved due to its tamper-proof ledger and decentralized system corroborating with the study in [13].

Therefore, Agentic AI system is shown to have a higher level of security and privacy, which would be crucial for applications that require a higher level of security and trust, such as financial or healthcare systems. However, the computing effort for the Agentic AI system is also high due to the blockchain operations. Potential scalability bottlenecks are highlighted by resource usage, which is higher for Agentic AI systems as they require a higher level of optimization, e.g., layer 2 solutions [14]. The non-Agentic AI system is efficient due to its simplicity, offering lower latency and sustained throughput. The non-Agentic AI system is best suited for high-volume scenarios that require a lower level of security. However, one problem with the non-Agentic AI system is its susceptibility to breaches and a single point of failure due to its centralized nature, which limits its applicability in a privacy-sensitive context. Although the error rates for both systems are comparable, i.e., 6% and 7%, it should be noted that the Agentic AI system errors are due to verification timeouts, while the errors of the non-Agentic AI system are due to failures in authentication. We also conducted experiments with a federated identity system (OAuth 2.0) under a load of 500 agents. Federated authentication achieved 88% success in security validation, which is higher than non-Agentic AI (80%) but lower than the Agentic AI (93%). However, it resulted in a verification latency of 220-350 ms (compared to 480 ms for Agentic AI) due to the token exchange overhead. The federated throughput (12-15 transmissions/sec) outperformed Agentic AI, but remained below that of centralised systems. Crucially, federated systems are prone to IdP outages and uptime drops to 92% during peak loads.

Challenges and Future Work

The integration of LLM into blockchain-based SSI systems poses several challenges that need to be overcome to fully realise its potential. One of the biggest challenges is the computational overhead associated with blockchain transactions. The process of verifying and recording transactions on the blockchain can be resource-intensive, which can slow down the performance of LLM agents. Optimizing blockchain protocols and exploring more efficient consensus mechanisms are crucial steps to reduce this overhead. Scalability is another major challenge, particularly when integrated with agentic AI environments where the number of agents and credential transactions can grow rapidly. Our experimental analysis highlights this issue, with the observed identity creation latency rising up to 600 ms under heavy load conditions (i.e., 500 agents). Such delays could hinder real-time multi-agent interactions, especially in high-throughput environments like

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Metric	System Type	10	50	100	250	500
Identity Creation Latency **(ms)**	Agentic AI	150	180	250	400	600
	Non-Agentic AI	50	55	60	70	80
Verification Latency **(ms)**	Agentic AI	100	130	200	320	480
	Non-Agentic AI	30	35	40	45	50
	Federated (OAuth 2.0)	-	-	-	-	350
Security Validation Success Rate (%)	Agentic AI	100	100	99	96	93
	Non-Agentic AI	98	95	90	85	80
	Federated (OAuth 2.0)	-	-	-	-	88
Unauthorized Access Blocked	Agentic AI	10/10	50/50	99/100	240/250	465/500
	Non-Agentic AI	9/10	47/50	90/100	212/250	400/500
Transaction Throughput (transmissions/second)						
Data Access	Agentic AI	24	19	14	9	5
	Non-Agentic AI	40	38	35	30	25
Financial Transfer	Agentic AI	19	15	11	7	3
	Non-Agentic AI	35	33	30	25	20
Error Rate (%)	Agentic AI	0	0	1	3	6
	Non-Agentic AI	1	2	3	5	7
Resource Utilization (%)						
CPU (Server)	Agentic AI	20	35	50	80	95
	Non-Agentic AI	5	10	15	20	25
Memory (Server)	Agentic AI	25	40	60	85	98
	Non-Agentic AI	10	15	20	25	30
System Uptime (%)	Agentic AI	100	100	99	98	95
	Non-Agentic AI	100	99	98	97	95
	Federated (OAuth 2.0)	-	-	-	-	92

TABLE 3. Experimental Results and Comparative Analysis for Agentic AI vs Non-Agentic AI Systems Under Varying Loads.

network management, smart cities, or industrial IoT. To address these scalability limitations, several Layer 2 scaling techniques can be considered. Layer 2 solutions aim to offload transaction processing from the main blockchain, improving transaction throughput and latency without sacrificing security. To reduce blockchain overhead, future implementations could combine state channels for real-time agent interactions, zk-Rollups for efficient batch credential verification [15] and optimised consensus algorithms such as Proof-of-Authority to reduce latency in permissioned environments. These techniques together could reduce credential issuance and verification times by up to 70–90% and enable the system to process tens of thousands of agent authentications per second without bottlenecks.

The implementation of blockchain-based SSI for LLM agents requires substantial infrastructure investment and ongoing maintenance, posing adoption challenges. Future work can target real-world deployments in various sectors (finance, healthcare, and industrial IoT), focusing on performance, scalability, and security, and include empirical comparisons with federated identity systems (e.g., OAuth 2.0, SAML). In these environments, deployment should follow a phased approach, beginning with 10-50 agent pilots to validate authentication and identify bottlenecks, considering Hyperledger Indy integration, existing identity systems, and secure agent-blockchain communication.

Exploration of ethical safeguards (e.g., user consent, identity recovery) and compliance with regulations such as GDPR, eIDAS, and HIPAA is also of interest. While current experiments validated the framework with Hyperledger Indy and simulated agents, multi-ledger interoperability (Ethereum for public VCs, Polygon for layer-2 scaling), industrial IoT testbeds (e.g., robotic process automation agents), and cross-domain healthcare collaborations via Hedera Hashgraph can be used to evaluate scalability under diverse real-world conditions.

Conclusion

In this paper, we presented a layered and modular architecture that integrates LLM agents with blockchain-based SSI, supported by a functional interface taxonomy and comparative evaluation.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Blockchain-based SSI ensured secure storage and controlled sharing of data, reducing the risk of security breaches while providing transparent and verifiable proof of identity. Our experimental results confirmed that the integration of LLM agents with blockchain-based SSI significantly improved security, with a validation success rate of 100% at low load and 93% at peak load, outperforming traditional centralized authentication systems that dropped to 80% under stress. However, the higher blockchain overhead resulted in higher identity creation latency (600ms vs. 80ms) and lower transaction throughput (5 vs. 25 transmissions/sec for data access), highlighting the limits of scalability, which calls for further research in this field.

References

[1] V. Alto, *Modern Generative AI With ChatGPT and OpenAI Models: Leverage the Capabilities of OpenAI's LLM for Productivity and Innovation With GPT3 and GPT4*. Birmingham, U.K.: Packt Publishing Ltd, 2023.

[2] M. A. K. Raiaan et al., "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," *IEEE Access*, vol. 12, pp. 26839–26874, 2024.

[3] V. Pallagani et al., "On the prospects of incorporating large language models (LLMs) in automated planning and scheduling (APS)," in *Proc. Int. Conf. Automated Planning Scheduling*, vol. 34, May 2024, pp. 432–444.

[4] Y. Cheng et al., "Exploring large language model based intelligent agents: Definitions, methods, and prospects," 2024, *arXiv:2401.03428*.

[5] W. Chen et al., "Internet of Agents: Weaving a web of heterogeneous agents for collaborative intelligence," 2024, *arXiv:2407.07061*.

[6] Y. Yao et al., "A survey on large language model (LLM) security and privacy: The good, the bad, and the Ugly," *High-Confidence Comput.*, vol. 4, no. 2, Jun. 2024, Art. no. 100211.

[7] F. He et al., "The emerged security and privacy of LLM agent: A survey with case studies," 2024, *arXiv:2407.19354*.

[8] C. Mazzocca et al., "A survey on decentralized identifiers and verifiable credentials," 2024, *arXiv:2402.02455*.

[9] M. R. Ahmed et al., "Blockchain-based identity management system and self-sovereign identity ecosystem: A comprehensive survey," *IEEE Access*, vol. 10, pp. 113436–113481, 2022.

[10] T. Sultana et al., "Data sharing system integrating access control mechanism using blockchain-based smart contracts for IoT devices," *Appl. Sci.*, vol. 10, no. 2, p. 488, Jan. 2020.

[11] F. Alzhrani, K. Saeedi, and L. Zhao, "Architectural patterns for blockchain systems and application design," *Appl. Sci.*, vol. 13, no. 20, p. 11533, Oct. 2023.

[12] R. Douceur, "The Sybil attack," in *Proc. 1st Int. Workshop Peer Peer Syst.* Cham, Switzerland: Springer, 2002, pp. 251–260.

[13] E. Zeydan et al., "Blockchain-based self-sovereign identity for federated learning in vehicular networks," in *Proc. 19th Int. Conf. Netw. Service Manage. (CNSM)*, Oct. 2023, pp. 1–7.

[14] K. Holley and M. Mathur, *LLMs and Generative AI for Healthcare: The Next Frontier*. Sebastopol, CA, USA: O'Reilly Media, 2024.

[15] S. Chaliasos et al., "Analyzing and benchmarking ZK-rollups," in *Proc. 6th Conf. Adv. Financial Technol. (AFT)*, 2024, p. 6.

Biographies

Abdullah Aydeger (aaydeger@fit.edu) is currently an Assistant Professor with the Electrical Engineering and Computer Science Department, Florida Institute of Technology.

Engin Zeydan (engin.zeydan@cttc.cat) is currently a Senior Researcher at the Centre Tecnològic de Telecomunicacions de Catalunya.

Josep Mangues-Bafalluy (josep.mangues@cttc.cat) is currently a Senior Researcher and the Head of the Services as Networks Research Unit, CTTC.

Yekta Turk (yektaturk@aselsan.com) is with Aselsan Corporation, working in the areas of networking and security.

Sunder Ali Khowaja (sunder.ali@ieee.org) is with the School of Computing, Dublin City University, and the ADAPT Centre, Dublin, Ireland, working in the area of agentic AI, AI governance, AI auditability, and privacy-preservation machine learning.

Kapal Dev (kapal.dev@ieee.org) is with Munster Technological University, Ireland, working in the areas of security and privacy of next-generation networks.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:18:44 UTC from IEEE Xplore. Restrictions apply.